

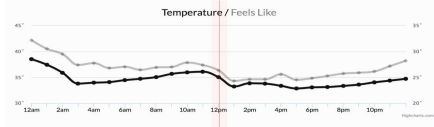
Time Series Forecasting

Zhirong Yang

Department of Computer Science NTNU

Time series data - chronological order

- Time series = a sequence of data points indexed in time order



Time	ActivePower	Ambient-Temperature	Nacelle-Temperature
2023-08-06 07:10:00	3.26093	9.93000	23.00000
2023-08-06 07:20:00	4.08909	9.92167	23.00000
2023-08-06 07:30:00	3.40836	9.90901	23.00000
2023-08-06 07:40:00	5.95700	10.81299	23.00000
2023-08-06 07:50:00	7.48506	10.35333	23.00000
2023-08-06 08:00:00	4.84470	10.15434	23.00000
2023-08-06 08:10:00	6.62998	9.89500	23.00000
2023-08-06 08:20:00	4.76331	9.93834	23.00000
2023-08-06 08:30:00	5.70828	10.32687	23.00000
2023-08-06 08:40:00	4.86776	10.39184	23.00000
2023-08-06 08:50:00	5.50134	10.76066	23.00000
2023-08-06 09:00:00	7.88312	10.55090	23.25566
2023-08-06 09:10:00	5.20189	11.00000	23.89667
2023-08-06 09:20:00	6.42136	11.27813	23.99813
2023-08-06 09:30:00	4.40971	11.24066	24.00000
2023-08-06 09:40:00	6.11000	11.59787	24.00000
2023-08-06 09:50:00	3.95668	11.00000	24.00000
2023-08-06 10:00:00	5.56489	11.00000	24.00000
2023-08-06 10:10:00	5.71667	10.88167	24.00000
2023-08-06 10:20:00	5.08827	10.99434	24.00000
2023-08-06 10:30:00	3.67998	10.21000	24.00000
2023-08-06 10:40:00	3.36189	10.58333	24.00000
2023-08-06 10:50:00	8.55930	10.77233	24.00000
2023-08-06 11:00:00	5.69966	11.00000	24.00000

Time series data - time intervals

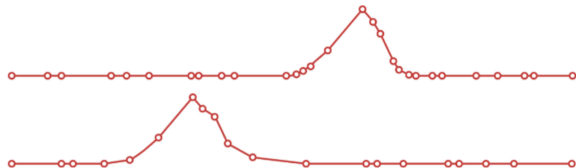
Metrics (Regular)

Measurements gathered at regular time intervals



Events (Irregular)

Measurements gathered at irregular time intervals



Time series examples - everywhere



Meteorology: weather variables, like temperature, pressure, wind



Economy and finance: economic factors (GNP), financial indexes, exchange rate, spread



Marketing: activity of business, sales, customers' experience, budget estimation



Industry: electric load, power consumption, voltage, sensors



Biomedicine: physiological signals (EEG), heart-rate, patient temperature



Internet of Things: sensor data prediction, data losses, device replacement



Performance and health monitoring: App performance, button clicks, server metrics, network data



Social sciences: unemployment rates, birth rates, population, migration data, political indicators



Epidemiology: disease rates, mortality rates, mosquito populations

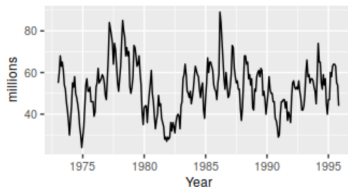
Challenges of time series for ML

- ▶ **Order matters**
- ▶ **Non-stationarity:** statistical properties (e.g., mean, variance) can change over time
- ▶ **Multiscale characteristics**
 - ▶ Trend: long-term increase or decrease in the data
 - ▶ Seasonality: repeating patterns at fixed frequencies (e.g., daily, yearly seasonality)
 - ▶ Cyclicity: cyclic patterns not tied to a fixed calendar frequency (e.g., economic cycles)
 - ▶ Noise: random variation
- ▶ **Evaluation complexity:** Must use time-based splits (train on past, test on future), making cross-validation non-trivial

Time series examples

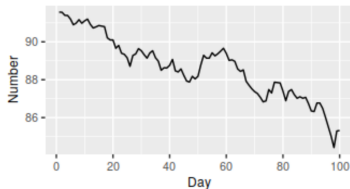
Seasonal + Cyclic

Sales of new one-family houses, USA

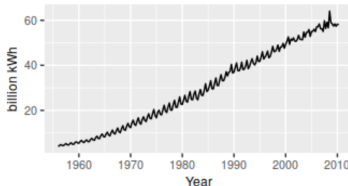


Trend

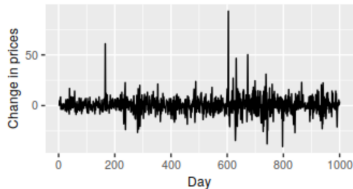
US treasury bill contracts



Australian quarterly electricity production



Google daily changes in closing stock price

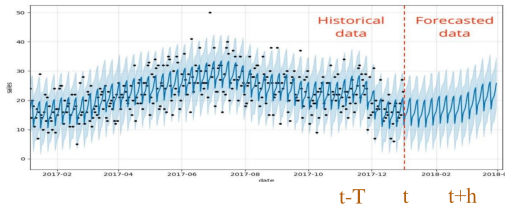
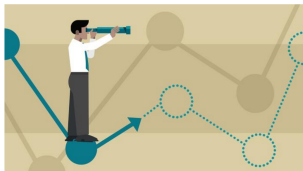


Trend + Seasonal

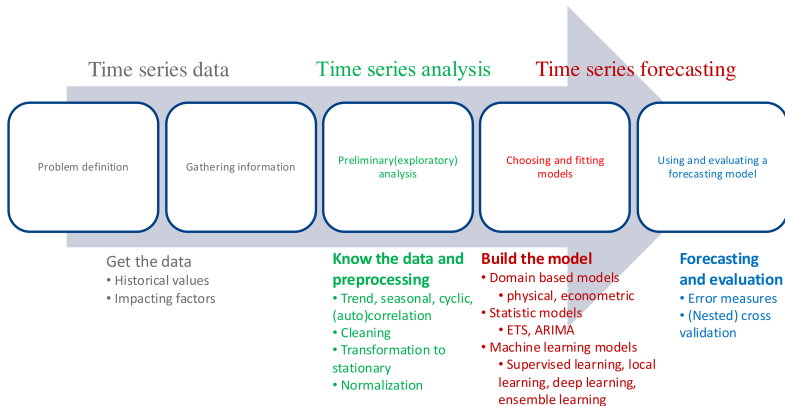
Stationary

Time series forecasting

- ▶ Stand in the **present** and forecast the **future** based on the **past** time series data
 - ▶ Forecasting can be one step or/and multi-steps ahead
 - ▶ Uncertainty is important
 - ▶ point forecasting: output mean/expectation
 - ▶ probabilistic forecasting: output prediction distribution



Workflow



The procedure can be loopy.

Time series analysis vs. forecasting

- ▶ Time series analysis = EDA
 - ▶ to understand the data
 - ▶ to identify patterns e.g. trends, cycles, and seasonal
 - ▶ to extract meaningful statistics and characteristics of the data
- ▶ Time series forecasting = making prediction
 - ▶ to select a model
 - ▶ to fit the model
 - ▶ to make and evaluate the predictions

Classical approaches

- ▶ They are not machine learning methods
- ▶ They are still popular
 - ▶ ETS
 - ▶ ARIMA
- ▶ Try to get its flavor
 - ▶ for future teamwork
 - ▶ for hybrid solutions
 - ▶ no need to digest all technical details

Classical approaches (statistical models)

1. Select a model family, e.g.,

ETS (Error-Trend-Seasonality) models

ADDITIVE ERROR MODELS

Trend	Seasonal		
	N	A	M
N	$y_t = \ell_{t-1} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \alpha \varepsilon_t$	$y_t = \ell_{t-1} + s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \alpha \varepsilon_t$ $s_t = s_{t-m} + \gamma \varepsilon_t$	$y_t = \ell_{t-1} s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \alpha \varepsilon_t / s_{t-m}$ $s_t = s_{t-m} + \gamma \varepsilon_t / \ell_{t-1}$
A	$y_t = \ell_{t-1} + b_{t-1} + \varepsilon_t$ $\ell_t = \ell_{t-1} + b_{t-1} + \alpha \varepsilon_t$ $b_t = b_{t-1} + \beta \varepsilon_t$	$y_t = \ell_{t-1} + b_{t-1} + s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + b_{t-1} + \alpha \varepsilon_t$ $b_t = b_{t-1} + \beta \varepsilon_t$ $s_t = s_{t-m} + \gamma \varepsilon_t$	$y_t = (\ell_{t-1} + b_{t-1}) s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + b_{t-1} + \alpha \varepsilon_t / s_{t-m}$ $b_t = b_{t-1} + \beta \varepsilon_t / s_{t-m}$ $s_t = s_{t-m} + \gamma \varepsilon_t / (\ell_{t-1} + b_{t-1})$
A_d	$y_t = \ell_{t-1} + \phi b_{t-1} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \phi b_{t-1} + \alpha \varepsilon_t$ $b_t = \phi b_{t-1} + \beta \varepsilon_t$	$y_t = \ell_{t-1} + \phi b_{t-1} + s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \phi b_{t-1} + \alpha \varepsilon_t$ $b_t = \phi b_{t-1} + \beta \varepsilon_t$ $s_t = s_{t-m} + \gamma \varepsilon_t$	$y_t = (\ell_{t-1} + \phi b_{t-1}) s_{t-m} + \varepsilon_t$ $\ell_t = \ell_{t-1} + \phi b_{t-1} + \alpha \varepsilon_t / s_{t-m}$ $b_t = \phi b_{t-1} + \beta \varepsilon_t / s_{t-m}$ $s_t = s_{t-m} + \gamma \varepsilon_t / (\ell_{t-1} + \phi b_{t-1})$

MULTIPLICATIVE ERROR MODELS

Trend	Seasonal		
	N	A	M
N	$y_t = \ell_{t-1}(1 + \varepsilon_t)$ $\ell_t = \ell_{t-1}(1 + \alpha \varepsilon_t)$	$y_t = (\ell_{t-1} + s_{t-m})(1 + \varepsilon_t)$ $\ell_t = \ell_{t-1} + \alpha(\ell_{t-1} + s_{t-m})\varepsilon_t$ $s_t = s_{t-m} + \gamma(\ell_{t-1} + s_{t-m})\varepsilon_t$	$y_t = \ell_{t-1} s_{t-m}(1 + \varepsilon_t)$ $\ell_t = \ell_{t-1}(1 + \alpha \varepsilon_t)$ $s_t = s_{t-m}(1 + \gamma \varepsilon_t)$
A	$y_t = (\ell_{t-1} + b_{t-1})(1 + \varepsilon_t)$ $\ell_t = (\ell_{t-1} + b_{t-1})(1 + \alpha \varepsilon_t)$ $b_t = b_{t-1} + \beta(\ell_{t-1} + b_{t-1})\varepsilon_t$	$y_t = (\ell_{t-1} + b_{t-1} + s_{t-m})(1 + \varepsilon_t)$ $\ell_t = \ell_{t-1} + b_{t-1} + \alpha(\ell_{t-1} + b_{t-1} + s_{t-m})\varepsilon_t$ $b_t = b_{t-1} + \beta(\ell_{t-1} + b_{t-1} + s_{t-m})\varepsilon_t$ $s_t = s_{t-m} + \gamma(\ell_{t-1} + b_{t-1} + s_{t-m})\varepsilon_t$	$y_t = (\ell_{t-1} + b_{t-1}) s_{t-m}(1 + \varepsilon_t)$ $\ell_t = (\ell_{t-1} + b_{t-1})(1 + \alpha \varepsilon_t)$ $b_t = b_{t-1} + \beta(\ell_{t-1} + b_{t-1})\varepsilon_t$ $s_t = s_{t-m}(1 + \gamma \varepsilon_t)$
A_d	$y_t = (\ell_{t-1} + \phi b_{t-1})(1 + \varepsilon_t)$ $\ell_t = (\ell_{t-1} + \phi b_{t-1})(1 + \alpha \varepsilon_t)$ $b_t = \phi b_{t-1} + \beta(\ell_{t-1} + \phi b_{t-1})\varepsilon_t$	$y_t = (\ell_{t-1} + \phi b_{t-1} + s_{t-m})(1 + \varepsilon_t)$ $\ell_t = \ell_{t-1} + \phi b_{t-1} + \alpha(\ell_{t-1} + \phi b_{t-1} + s_{t-m})\varepsilon_t$ $b_t = \phi b_{t-1} + \beta(\ell_{t-1} + \phi b_{t-1} + s_{t-m})\varepsilon_t$ $s_t = s_{t-m} + \gamma(\ell_{t-1} + \phi b_{t-1} + s_{t-m})\varepsilon_t$	$y_t = (\ell_{t-1} + \phi b_{t-1}) s_{t-m}(1 + \varepsilon_t)$ $\ell_t = (\ell_{t-1} + \phi b_{t-1})(1 + \alpha \varepsilon_t)$ $b_t = \phi b_{t-1} + \beta(\ell_{t-1} + \phi b_{t-1})\varepsilon_t$ $s_t = s_{t-m}(1 + \gamma \varepsilon_t)$

N=None, A=Additive, M=Multiplicative, A_d=Additive damped

- by eyes, or
- by information criterion (e.g., AIC or BIC); software available

2. Fit the model paramters

ARIMA (AutoRegressive Integrated Moving Average)

- ▶ ARIMA is a popular statistical model for time series
- ▶ It captures three key aspects of a series:
 - ▶ Autoregression, $AR(p)$: modeling the **lagged** information

$$X_t = c + \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + \epsilon_t$$

- ▶ Integration (I): differencing the data to make it **stationary**.
 - ▶ $d = 1$: $\Delta X_t = X_t - X_{t-1}$.
 - ▶ $d = 2$: $\Delta^2 X_t = \Delta X_t - \Delta X_{t-1} = X_t - 2X_{t-1} + X_{t-2}$
 - ▶ $\Delta^d X_t = \sum_{i=0}^d (-1)^i \binom{d}{i} X_{t-i}$
- ▶ Moving Average, $MA(q)$: modeling **residual error** with a moving average for lagged observations

$$X_t = c + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

- ▶ Putting together, $ARIMA(p, d, q)$:

$$\Delta^d X_t - \sum_{i=1}^p \phi_i \Delta^d X_{t-i} = c + \epsilon_t + \sum_{j=1}^q \theta_j \epsilon_{t-j}$$

Recipe for building an ARIMA model

1. Estimate (p, d, q)

- ▶ Plot the series and look for constant mean/variance.
- ▶ Use statistical tests (e.g., Augmented Dickey–Fuller) to confirm non-stationarity.
- ▶ Difference the series d times until it appears stationary.
- ▶ Sharp cutoff in ACF (Autocorrelation Function) after lag q
- ▶ Sharp cutoff in PACF (Partial ACF) after lag p

2. Estimate ϕ and θ

- ▶ use maximum likelihood (or least squares)
- ▶ use `statsmodels` in Python, `forecast` in R

3. Diagnostic checking

- ▶ Analyze residuals: they should behave like white noise (no remaining autocorrelation).
- ▶ Plot residual ACF and conduct Ljung-Box test.
- ▶ If diagnostics fail, revisit (p, d, q) choices.

An ARIMA Demo Notebook

Classical methods: pros and cons

- ▶ Pros:
 - ▶ Good statistical theory (confidence intervals, etc.).
 - ▶ Good for many simple seasonal patterns
 - ▶ Good when data is limited
 - ▶ Often interpretable (e.g., AR coefficients, seasonal indices).
- ▶ Cons:
 - ▶ Usually assume linear relationships (bad for complex patterns).
 - ▶ Need stationary data; otherwise require transformations like differencing, detrending
 - ▶ Bad for high dimensionality
 - ▶ Difficult for ensemble prediction.
 - ▶ Selecting (p, d, q) can be time-consuming
- ▶ Classical methods serve as good baselines; we often compare ML models against ARIMA or naive forecasts.

Machine Learning for Time Series

- ▶ Why ML/DL for Time Series?
 - ▶ to capture complex nonlinear patterns, interactions, and long-range dependencies
 - ▶ easier for multiple correlated time series or additional features
- ▶ Two broad approaches:
 - ▶ **Feature-based ML:**
 1. feature extraction (convert time series to tabular)
 2. apply tabular ML methods (e.g., random forests, XGBoost)
 - ▶ **Sequence modeling:** use models that inherently handle sequence data (recurrent neural networks, convolutional networks, Transformers).
- ▶ However, ML models often require more data and computational resources

Feature-based ML for Time Series

Preprocessing - sliding window

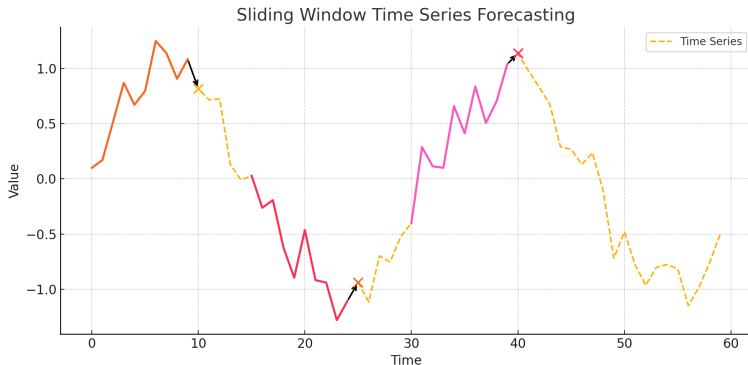
- ▶ Many machine learning methods require vector inputs
- ▶ Some others require a sequence of vectors (e.g., Transformer)
- ▶ Need to convert time series to (a sequence of) vectors
- ▶ Example (windows size=4)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-0.3	2.1	1.7	-2.0	0.8	-1.1	0.6	-0.7	2.1	-1.3	1.0	0.2	0.8	-0.5	0.8

4	5	6	7	8	9	10	11	12	13	14	15
-0.3	2.1	1.7	-2.0	0.8	-1.1	0.6	-0.7	2.1	-1.3	1.0	0.2
2.1	1.7	-2.0	0.8	-1.1	0.6	-0.7	2.1	-1.3	1.0	0.2	0.8
1.7	-2.0	0.8	-1.1	0.6	-0.7	2.1	-1.3	1.0	0.2	0.8	-0.5
-2.0	0.8	-1.1	0.6	-0.7	2.1	-1.3	1.0	0.2	0.8	-0.5	0.8

- ▶ Notice!
 - ▶ The intervals between time steps must be even (regular)
 - ▶ The vectors by sliding windows are not independent

Sliding-window training samples



Fit a model to predict a value with the window (size=10) before it

- ▶ Window size is a hyperparameter
 - ▶ Too small windows $\Rightarrow$ lose information
 - ▶ Too large windows $\Rightarrow$ too complex patterns, not enough samples

Time series features

- ▶ Extract various features from a time series, e.g.,
 - ▶ Summary statistics
 - ▶ mean, variance, percentiles, IQR, etc.
 - ▶ Autocorrelation and lag features
 - ▶ autocorrelation coef. up to lag k , Partial Autocorrelation (PACF)
 - ▶ Rolling and windowed statistics
 - ▶ Moving mean / std, rolling min / max, etc.
 - ▶ Trend and seasonality
 - ▶ trend slope, seasonal strength, residue statistics, etc.
 - ▶ Frequency-domain features
 - ▶ Fourier coefficients, power spectrum, wavelet transform coefficients, etc.
 - ▶ Shape and complexity metrics
 - ▶ peak / valley counts, signal-to-noise ratio (SNR), entropy, etc.
- ▶ Feature extraction tools: TSPfresh, TSFEL, tsflex, Kats, etc.
- ▶ Concatenate them (maybe with other data) as tabular data for ML

Sequence Modeling

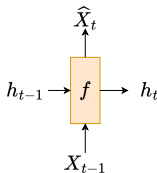
Recurrent Neural Networks (RNNs)

- ▶ One-step forecasting: $X_t \approx \hat{X}_t = f(X_1, \dots, X_{t-1}; \theta)$

Recurrent Neural Networks (RNNs)

- ▶ One-step forecasting: $X_t \approx \hat{X}_t = f(X_1, \dots, X_{t-1}; \theta)$
- ▶ However, variable-length input for f is difficult
- ▶ RNN summarizes the history up to t into a state h_t

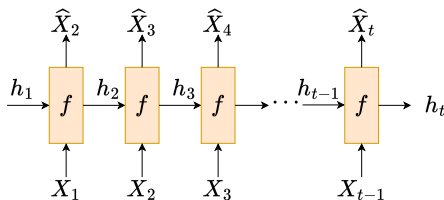
$$(h_t, \hat{X}_t) = f(h_{t-1}, X_{t-1}; \theta)$$



Recurrent Neural Networks (RNNs)

- ▶ One-step forecasting: $X_t \approx \hat{X}_t = f(X_1, \dots, X_{t-1}; \theta)$
- ▶ However, variable-length input for f is difficult
- ▶ RNN summarizes the history up to t into a state h_t

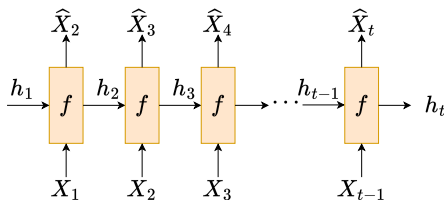
$$(h_t, \hat{X}_t) = f(h_{t-1}, X_{t-1}; \theta)$$



Recurrent Neural Networks (RNNs)

- ▶ One-step forecasting: $X_t \approx \hat{X}_t = f(X_1, \dots, X_{t-1}; \theta)$
- ▶ However, variable-length input for f is difficult
- ▶ RNN summarizes the history up to t into a state h_t

$$(h_t, \hat{X}_t) = f(h_{t-1}, X_{t-1}; \theta)$$



- ▶ Vanilla RNNs are difficult to train
 - ▶ gradients often vanish or explode
 - ▶ Pascanu et al. "On the difficulty of training recurrent neural networks" 2013

Long Short-Term Memory (LSTM)

A more dedicated RNN

Use gates to remember or to forget

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

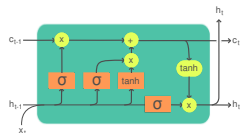
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$



Legend:

Layer

Componentwise

Copy

Concatenate



LSTM Demo

(Additional Reading) Transformer

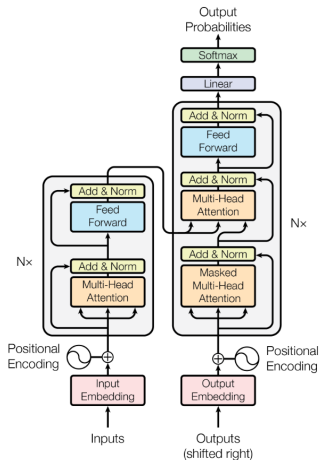
Given a sequence $X \in \mathbb{R}^{T \times d}$

$\text{MultiHead} = \text{Concatenate}(\text{head}_1, \dots, \text{head}_h) W^O$

$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V)$

$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$

- ▶ Transformer is good for big data
- ▶ But time series are often not big
- ▶ So, the original Transformer is usually not good for time series

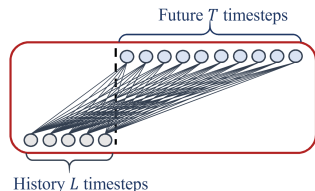


Decomposed Linear (DLinear)

- ▶ Zeng et al. *Are Transformers Effective for Time Series Forecasting?* 2022
- ▶ They proposed DLinear—embarrassingly simple but effective

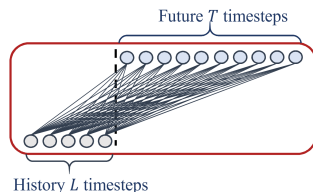
Decomposed Linear (DLinear)

- ▶ Zeng et al. *Are Transformers Effective for Time Series Forecasting?* 2022
- ▶ They proposed DLinear—embarrassingly simple but effective
- ▶ input $X \in \mathbb{R}^L$, output $\hat{Y} \in \mathbb{R}^T$
- ▶ Basic linear: $\hat{Y} = WX$



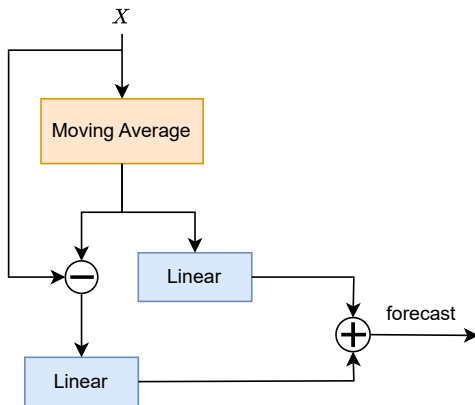
Decomposed Linear (DLinear)

- ▶ Zeng et al. *Are Transformers Effective for Time Series Forecasting?* 2022
- ▶ They proposed DLinear—embarrassingly simple but effective
- ▶ input $X \in \mathbb{R}^L$, output $\hat{Y} \in \mathbb{R}^T$
- ▶ DLinear decomposes $X = A + R$
 - ▶ A is a moving average of X :
$$A_i = \frac{1}{k} \sum_{j=0}^{k-1} X_{i-j}$$
 - ▶ R is the residual: $R = X - A$
- ▶ DLinear: $\hat{Y} = W_A A + W_R R$



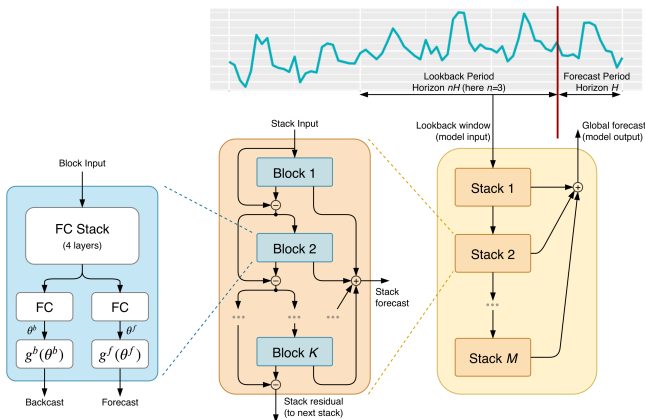
A DLinear Demo Notebook

Generalizing DLinear



We can replace the Moving Average and Linears with neural networks.

N-BEATS (Neural Basis Expansion Analysis for Interpretable Time Series Forecasting)



- For each block: input $\mathbf{x}_I$, outputs $\hat{\mathbf{x}}_I$ and $\hat{\mathbf{y}}_I$
 - $\mathbf{h}_{I,k} = \text{ReLU}(\mathbf{W}_{I,k}\mathbf{h}_{I,k-1} + \mathbf{b}_{I,k})$, for $k = 1, 2, 3, 4$ with $\mathbf{h}_{I,0} = \mathbf{x}_I$
 - $\theta_I^b = \mathbf{W}_I^b \mathbf{h}_{I,4}$, $\theta_I^f = \mathbf{W}_I^f \mathbf{h}_{I,4}$
 - $\hat{\mathbf{x}}_I = \theta_I^b \mathbf{v}_I^b + \mathbf{b}_I^b$, $\hat{\mathbf{y}}_I = \theta_I^f \mathbf{v}_I^f + \mathbf{b}_I^f$
- For each stack: $\mathbf{x}_I = \mathbf{x}_{I-1} - \hat{\mathbf{x}}_{I-1}$, $\hat{\mathbf{y}} = \sum_I \hat{\mathbf{y}}_I$

(Additional Reading) Interpretable architecture

- Generic architecture

$$\hat{\mathbf{y}}_l = \theta_l^f \mathbf{v}_l^f + \mathbf{b}_l^f$$

- Interpretable architectures

- Trend model

$$\hat{\mathbf{y}}_{s,l} = \sum_{i=0}^p \theta_{s,l,i}^f t^i,$$

where $\mathbf{t} = [0, 1, 2, \dots, H-2, H-1]^T / H$

- Seasonality model

$$\hat{\mathbf{y}}_{s,l} = \sum_{i=0}^{\lfloor H/2-1 \rfloor} \theta_{s,l,i}^f \cos(2\pi it) + \theta_{s,l,i+\lfloor H/2 \rfloor}^f \sin(2\pi it)$$

An N-BEATS Demo Notebook

(Additional Reading) Recent work

- ▶ N-HiTS: Hierarchical interpolation & multi-rate sampling for long-term forecasting
- ▶ DLinear: Direct linear decomposition model outperforming many Transformers
- ▶ Informer: ProbSparse self-attention for efficient long-sequence forecasting
- ▶ Autoformer: In-block decomposition & Auto-Correlation for trend/seasonality separation
- ▶ PatchTST: Channel-independent patching tokens for multivariate scaling & self-supervised pre-training
- ▶ DARTS Python library

Evaluation Metrics for Forecasting

- ▶ **Mean Squared Error (MSE):** $\frac{1}{N} \sum_{t=1}^N (y_t - \hat{y}_t)^2$.
- ▶ **(RMSE):** square root of MSE
 - ▶ MSE and RMSE are sensitive to outliers
- ▶ **Mean Absolute Error (MAE):** $\frac{1}{N} \sum_{t=1}^N |y_t - \hat{y}_t|$
 - ▶ MSE, RMSE and MAE $\in [0, \infty)$
- ▶ **Mean Absolute Percentage Error (MAPE):**
 $\frac{1}{N} \sum_{t=1}^N \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100\%$
 - ▶ can be problematic if y_t is close to zero
- ▶ **Symmetric MAPE (sMAPE):** $\frac{1}{N} \sum_{t=1}^N \frac{|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|} \times 100\%$
 - ▶ forecasting competitions often use sMAPE

Train/Test Splitting and Validation

- ▶ Avoid random shuffle splitting
- ▶ Test set should cover relevant seasons if seasonality exists

Sliding Window



Expanding Window



Forecasting competitions

- ▶ Represents the SOTA and methodology trend
- ▶ M4 Competition (2018)
 - ▶ Domains: Micro, Macro, Industry, Finance, Demographic
 - ▶ Intervals: Yearly, Quarterly, Monthly, Weekly, Daily, Hourly
 - ▶ Winner: ES-RNN (exponential smoothing + RNN)
 - ▶ N-BEATS improves a lot after the competition
- ▶ M5 Competition (2020)
 - ▶ Estimate the unit sales of Walmart retail goods
 - ▶ Winner: LightGBM
- ▶ M6 Competition (2022-2023)
 - ▶ 50 stocks from the S&P 500 index and 50 international exchange-traded funds
 - ▶ Overall winner: encoder-decoder neural network + meta-learning
 - ▶ Return (IR) performance winner: AutoTS (AutoML for TS)
- ▶ Other competitions: GEFCom series and Kaggle/industry challenges